

SRTP Simple Reliable Transport Protocol

Projet 1 · LINFO1341 Réseaux informatiques

Belojev Abubakar (NOMA - 23442301)

Zhabrailov Amerkhan (NOMA - 34122200)

Vue d'ensemble

Ce rapport présente notre implémentation du protocole SRTP pour le projet 1 de LINFO1341. Le but est de transférer des fichiers de manière fiable entre un serveur et un client, en utilisant UDP/IPv6 avec une fenêtre glissante.

- **Langage:** Python 3
- **Transport:** UDP · IPv6
- **Mécanisme:** Fenêtre glissante avec ACKs cumulatifs
- **Taille max payload:** 1024 octets
- **Taille de fenêtre:** 32 paquets (côté récepteur)

Architecture

Déroulement d'un transfert

- Le client envoie un paquet GET /chemin (seqnum=0, window=32)
- Le serveur acquitte le GET (seqnum=1)
- Le serveur découpe le fichier en chunks de 1024 octets et les envoie
- Le client acquitte chaque chunk reçu et indique la place restante dans son buffer
- Le serveur envoie un paquet vide (Length=0) pour indiquer la fin du fichier
- Le client acquitte la fin et sauvegarde le fichier sur le disque

Fichiers

- packet.py Encode et décode les paquets SRTP (header, CRC1, payload, CRC2)
- server.py Reçoit les requêtes GET et envoie le fichier avec la fenêtre glissante
- client.py Envoie le GET, réordonne les paquets hors-ordre et sauvegarde le fichier

Format des paquets SRTP

Champ	Taille (bits)	Description
Type	2	PTYPE_DATA=1, PTYPE_ACK=2, PTYPE_SACK=3
Window	6	Place disponible dans le buffer du récepteur (0–63)
Length	13	Taille du payload en octets (0–1024)
Seqnum	11	Numéro de séquence (modulo 2048)
Timestamp	32	time.monotonic() en ms — copié dans les ACKs
CRC1	32	CRC32 du header + timestamp
Payload	0–1024 octets	Données (absent si Length=0)
CRC2	32	CRC32 du payload (absent si Length=0)

Champ Timestamp

Qu'est ce qu'on met dans le Timestamp ?

On stocke la valeur de `time.monotonic()` en millisecondes, tronquée à 32 bits. On utilise une horloge monotone (et non `time.time()`) pour éviter tout problème si l'heure système change pendant le transfert.

Ce qu'on n'a pas fait avec

Dans notre implémentation, le RTT est calculable mais on ne s'en sert pas : le timeout de retransmission reste fixe à 3 secondes. On aurait pu l'ajuster dynamiquement en fonction du RTT mesuré, mais on a préféré garder quelque chose de simple et prévisible.

Comment on l'utilise ?

Le récepteur copie le Timestamp du dernier paquet DATA reçu dans chacun de ses ACKs. L'émetteur peut ainsi calculer le RTT (aller-retour) pour chaque paquet en comparant la valeur reçue avec son horloge courante.

Timeout de retransmission

Valeur choisie

3 secondes Le cahier des charges indique que le réseau peut introduire jusqu'à 2000 ms de latence. On a donc choisi 3 secondes pour avoir une marge suffisante sans attendre trop longtemps en cas de perte réelle d'un paquet.

Limites de notre implémentation

Le timeout est fixe pour toute la durée du transfert. Idéalement, on pourrait le calculer à partir du RTT mesuré grâce au champ Timestamp, mais on a choisi de garder ça simple pour cette version.

Fonctionnement côté serveur

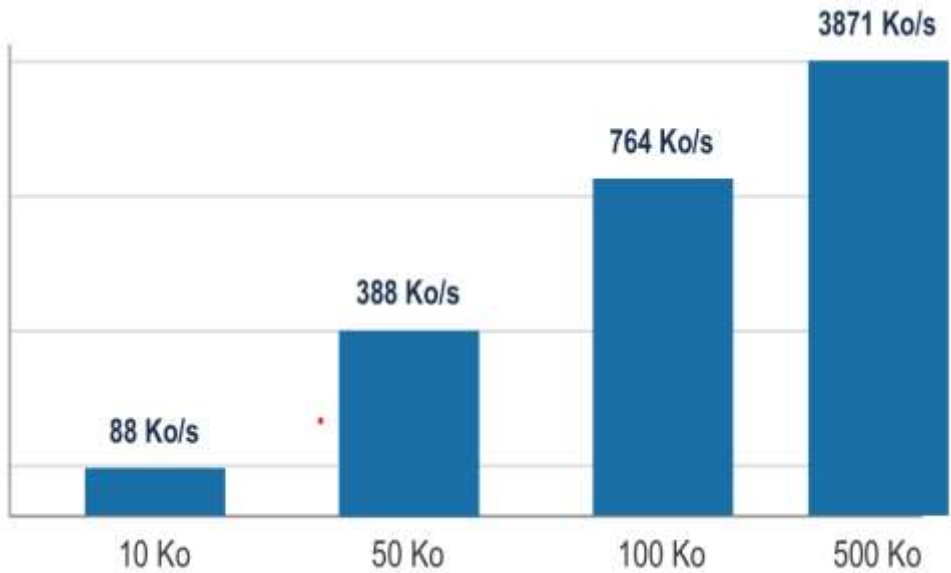
Le serveur vérifie périodiquement (toutes les 50 ms) si des paquets envoyés n'ont pas été acquittés depuis plus de 3 secondes. Si c'est le cas, il les retransmet.

Performances

Méthodologie

Mesures réalisées en loopback IPv6 (:::1) sur la même machine, sans perte ni délai artificiel. Pour chaque taille, on lance 3 transferts et on prend la moyenne. Le temps mesuré inclut l'envoi du GET, la réception de tous les chunks et la réception du paquet de fin.

Taille (Ko)	Chunks	Temps moyen (ms)	Débit (Ko/s)
10	10	112	88
50	49	126	388
100	98	128	764
500	489	126	3 871



Constat

Le débit augmente avec la taille car le temps de transfert reste quasi constant (~125 ms) : la fenêtre de 32 paquets permet d'envoyer presque tout le fichier d'un coup. L'overhead du GET initial devient donc proportionnellement plus faible sur les gros fichiers.

Partie critique — Fenêtre glissante

La partie la plus importante du code est la boucle d'envoi dans `send_file()` côté serveur. C'est là que se gère la fenêtre glissante et les retransmissions.

Ce qui limite les performances:

Attente de 50 ms entre chaque lecture d'ACK

Le serveur tourne en boucle avec un timeout de 50 ms sur `recvfrom()`. Si les ACKs arrivent vite, on attend quand même ces 50 ms à chaque tour.

Timeout fixe de 3 seconds

Si un paquet est perdu, on attend 3 secondes avant de le renvoyer. Un timeout calculé depuis le RTT réel serait plus efficace.

Un client à la fois

Le serveur traite une requête, puis attend la suivante. Plusieurs clients en même temps ne sont pas supportés.

Tests

Tests unitaires — test_packet.py (15 tests)

- Taille correcte d'un paquet DATA et d'un ACK
- Encode puis decode : tous les champs sont bien conservés
- Rejet si le CRC1 est corrompu (header modifié)
- Rejet si le CRC2 est corrompu (payload modifié)
- Rejet si le payload est tronqué
- Rejet si Length > 1024
- Erreur si type, window ou seqnum est hors bornes

Tests d'intégration — test_integration.py (4 tests)

- Transfert d'un petit fichier texte, vérifié octet par octet
- Transfert d'un fichier de 5 Ko
- Fichier inexistant : le client reçoit un fichier vide
- Deux transferts à la suite : le serveur reste disponible

Résultat : 19 tests · 19 réussis · 0 échec —
pytest tests/ -v

Acquittements sélectifs (SACK)

Ce qu'on a implémenté

Le type PTYPE_SACK=3 est défini dans packet.py et les ACKs sont bien décodés des deux côtés. Mais dans cette version, on n'envoie que des ACKs cumulatifs (PTYPE_ACK) : le client signale juste le prochain numéro de séquence attendu, sans indiquer les paquets reçus hors-ordre.

Ce que le SACK changerait

Avec les ACKs cumulatifs, si le paquet n°5 est perdu mais que les paquets 6, 7, 8... sont déjà arrivés, le serveur doit retransmettre tous les paquets à partir du n°5. Avec le SACK, le client pourrait dire "j'ai 6, 7, 8 mais pas 5", et le serveur ne retransmettrait que le n°5

Gain attendu

Le gain serait surtout visible sur des réseaux avec beaucoup de pertes. En loopback (réseau idéal, 0 perte), il n'y a pratiquement aucune différence car les retransmissions n'arrivent presque jamais

Conclusion

Ce qui est implémenté

- ✓ Encodage et décodage des paquets avec CRC1 et CRC2
- ✓ Fenêtre glissante côté serveur avec retransmission automatique ✓ Buffer côté client pour gérer les paquets qui arrivent dans le désordre
- ✓ Contrôle de flux : le client annonce la place libre dans son buffer
- ✓ Détection des paquets corrompus ou tronqués

Ce qu'on pourrait améliorer

- Calculer le timeout depuis le RTT réel plutôt qu'une valeur fixe
- Gérer plusieurs clients en même temps
- Ajouter des tests avec conditions réseau dégradées (pertes, corruption, latence artificielle)